

# **WAVE by Examples**

**Michael Scheer  
Helmholtz-Zentrum Berlin**

**October 26, 2025**

## Contents

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>I</b>  | <b>Introduction</b>                                    | <b>2</b>  |
| <b>II</b> | <b>Getting Started</b>                                 | <b>3</b>  |
| II.1      | Running WAVE . . . . .                                 | 3         |
| II.2      | Plotting with Python . . . . .                         | 4         |
| II.3      | The Graphical User Interface <i>WAVESHOP</i> . . . . . | 4         |
| II.4      | The Input File <i>wave.in</i> . . . . .                | 5         |
| <b>1.</b> | <b>Example: Selecting a Magnetic Field Model</b>       | <b>8</b>  |
| <b>2.</b> | <b>Example: Tracking through Magnetic Fields</b>       | <b>11</b> |
| <b>3.</b> | <b>Example: The Concept of Source Points</b>           | <b>15</b> |
| <b>4.</b> | <b>Example: APPLE-II Undulator</b>                     | <b>21</b> |
| <b>5.</b> | <b>Example: Simple Build-In Optimization</b>           | <b>24</b> |
| <b>6.</b> | <b>Example: "Real" Beams, Part I</b>                   | <b>28</b> |
| <b>7.</b> | <b>Example: Global Magnetic Field Options</b>          | <b>31</b> |
| <b>8.</b> | <b>Example: Magnetic Field Errors</b>                  | <b>35</b> |
| <b>9.</b> | <b>Example: Real Beams, Part II</b>                    | <b>39</b> |
|           | <b>References</b>                                      | <b>43</b> |

## I Introduction

WAVE has been developed at BESSY over the years to track electrons through magnetic fields and to calculate spontaneous synchrotron radiation [1]. This document illustrates the usage of WAVE and makes you familiar with it by following the given examples.

WAVE is mainly controlled by an ASCII input file named *wave.in*. For each example, there is a script *wave\_example\_n.sh* to create *wave.in* from *\$WAVE/input/wave.examples*, where *n* is the number of the example. Once *wave.in* is created by the corresponding script, you may compare *\$WAVE/input/wave.examples* and *wave.in* to see, which variables have changed.

There is a graphical user interface (GUI), a Python script called *WAVESHOP* to setup *wave.in*, to run WAVE, and to visualize the results. This is explained in the first example. However, not all options for WAVE are available in the GUI.

Once WAVE is installed, you should have a WAVE directory and shell variable WAVE pointing to it. It is assumed, that the user knows to edit a file and how to open a terminal, e.g. *bash* under Linux or powershell and under Windows. As an alternative one may run the Python shell *idle*.

WAVE and the related software is published under the GNU General Public License. To get WAVE, please contact the author or get it from github:

```
git clone https://github.com/MicScheer/wave.git wave
```

## II Getting Started

The examples in this document refer to a **LINUX** machine, however, they should be easy to understand and adapted to a **WINDOWS** computer.

The main differences are, that the slashes in the file and path names must be replaced by backslashes, shell scripts (\*.sh) must be replaced by \*.bat files, and the system variables \$V must be replaced by %V%.

### II.1 Running WAVE

First define a shell variable `$WAVE` containing the path to your WAVE -directory:

```
export WAVE=YOUR-WAVE-DIRECTORY
```

```
export WAVE_INCL=$WAVE
```

It is recommended to put these lines in a script, see e.g.

```
. $WAVE/shell/set_wave_environment.sh
```

and to excute it at the beginning of a WAVE -session.

To run WAVE the traditional way is to is to enter in a terminal

```
cd $WAVE/bin/stage
```

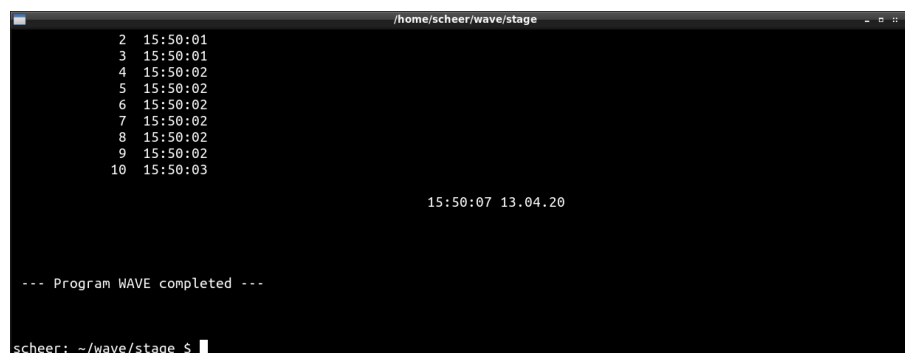
```
../bin/wave.exe
```

It is recommended to use the script

```
$WAVE/stage/wave
```

to run WAVE .

WAVE should run and write some output to your terminal (Fig. 1).

A terminal window titled "/home/scheer/wave/stage" showing the output of the WAVE program. The output consists of a list of numbers 2 through 10, each followed by a timestamp. The timestamps for lines 2-9 are 15:50:01 and for line 10 is 15:50:02. Below this list, the timestamp 15:50:07 13.04.20 is displayed. At the bottom, the text "--- Program WAVE completed ---" is shown. The prompt "scheer: ~/wave/stage \$" is visible at the bottom left.

```
/home/scheer/wave/stage
2 15:50:01
3 15:50:01
4 15:50:02
5 15:50:02
6 15:50:02
7 15:50:02
8 15:50:02
9 15:50:02
10 15:50:03

15:50:07 13.04.20

--- Program WAVE completed ---
scheer: ~/wave/stage $
```

Fig. 1: Terminal output of WAVE

The program writes some output to the screen and the output text files **wave.out** and **WAVE.mhb**. The file **wave.out** contains the most important in- and output of the run, while **WAVE.mhb** contains the histograms and n-tuples to be plotted. Please open these files and have a look...

If the program crashes, it must probably be recompiled in your environment by

```
python3 $WAVE/python/make_wave.py
```

if you have **gfortran** installed, or by

```
python3 $WAVE/python/make_wave_intel.py
```

if you have the Intel compiler **ifx** installed.

If these Python scripts are run with an argument, they run in a verbose mode.

## II.2 Plotting with Python

A convenient way to visualize the histograms and n-tuples is to use a Python scripts

```
python3 -i ../python/waveplot.py
```

First, we check the installation of Python. Enter:

```
python3
```

If you get a **Python** command prompt, **Python3** is installed. Note: Under **WINDOWS** the command is probably **python** instead of **python3**. However, the version 3.x of **Python** is required. You can leave the **Python** shell with:

```
quit()
```

If the test was successful, try the same with **ipython3**:

```
ipython3
```

If there is no command **ipython3**, you can install it (given **pip3** is installed):

```
pip3 install ipython3
```

## II.3 The Graphical User Interface **WAVESHOP**

There is a graphical user interfaces (GUI) based on **Python** and **Tkinter**.

To run the GUI **WAVESHOP** (Fig. 2), you need **python3** or **ipython3**. In addition, the file **waves.wvs** must be present in the working directory. The **Python** script **../python/waveshop.py** is started by the command alias **waveshop**. Thus enter

```
WAVESHOP
```

or, if the alias is not defined accordingly, by

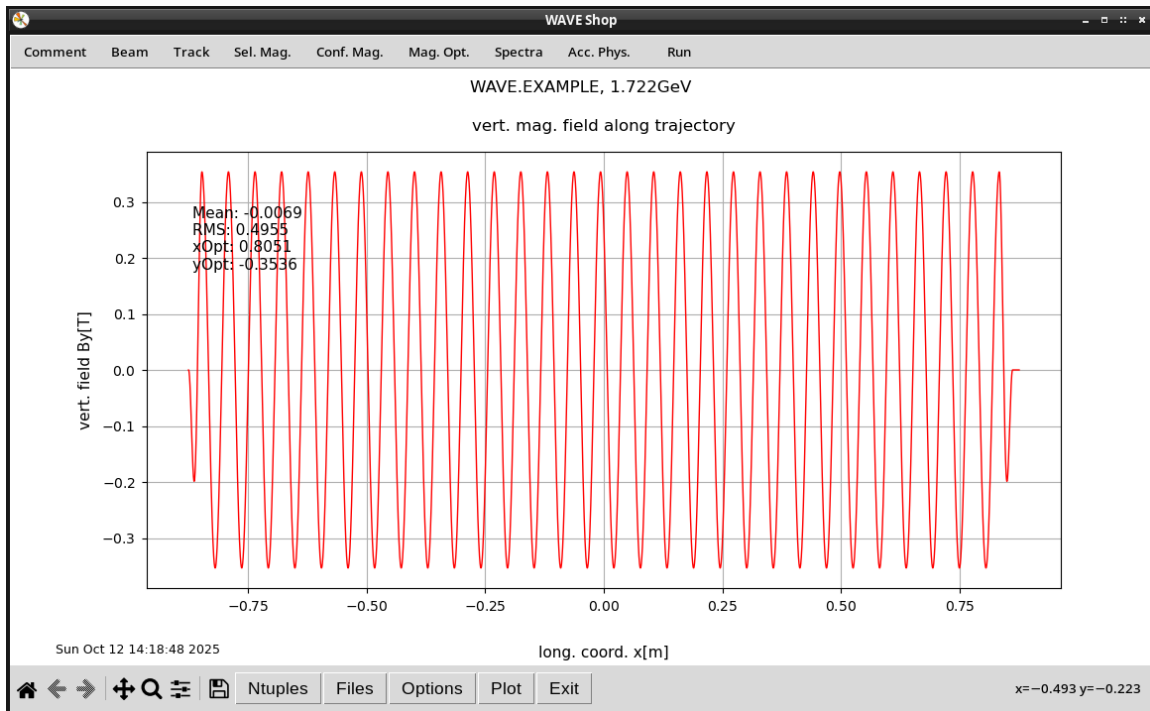


Fig. 2: Main window of *WAVESHOP*. The upper menu items are used to set-up *wave.in* and to run WAVE, the lower buttons to plot the results.

```
python3 -i ../python/waveshop.py
```

## II.4 The Input File *wave.in*

The GUI for WAVE covers many standard calculations, however, to get the most out of WAVE, it is necessary to study and understand the input file *wave.in*.

We open *wave.in* with an editor and search *\$CONTRL*. This keyword belongs to a *FORTRAN* namelist. A namelist begins with such a keyword and ends with *\$END*. The file *wave.in* is actually nothing than a list of namelists.

A namelist is a list of variables, read by the program. The position of the variable in the namelist doesn't matter. Missing variables are initialized as zero or an empty string. For the name of the variables the *FORTRAN* conventions apply, i.e. variables beginning with *I, J, K, L, M, N* are integer, all others are strings, floating point, or complex numbers. The integer must not have a decimal point, while the floating point and complex numbers must have one. The same variable may occur several times within a namelist. Then the last value is accepted. If a whole namelist occurs several times, only the first one is read. Comments must begin with "!".

The namelist *\$CONTRL* is the most important one of *wave.in*. It controls the program flow, the tracking, the field selection and so on.

```

F10 key ==> File Edit Search Buffers Windows System Help
!*****
!
!                               VERSION 3.03/00
!
!$CONTRL
!----- USER COMMENT -----
!
!   CODE='WAVE.EXAMPLE VERSION 3.03/00'
!
!----- MAIN MODES -----
!
!   ! The undulator and wiggler modes should work for standard
!   ! insertion devices. Reasonable settings for some parameters
!   ! are taken (mainly in namelist COLLIN).
!   ! Experienced users might prefer their own settings.
!
!   IUNDULATOR=1   ! UNDULATOR MODE:
!
!   ! IBUNCH.EQ.0:
!   ! whole trajectory is taken as source of
!   ! synchrotron light (ignoring input of
!
-----+ Jed: wave.in (Text) 58/3170 1 11:40am -----

```

Fig. 3: First part of the input file *wave.in*

A short example: Look for a line, containing

***KELLIP=1***

This variable activates an analytical model of an elliptical undulator. The comment in this line mentions the namelist ***\$ELLIPN***. Hence, we search ***\$ELLIPN***.

```

Xjed
F10 key ==> File Edit Search Buffers Windows System Help
!$ELLIPN   ! helical undulator
!
!   PARKELL=0.   ! if not zero, it is effective K-parameter of undulator
!   ! mag. fields B0ELLIPV and B0ELLIPH are adjusted
!   ! signs and ratio of B0ELLIPV and B0ELLIP are kept
!
!   B0ELLIPV=0.353553390593   ! vertical peak field [T]
!   B0ELLIPH=0.353553390593   ! horizontal peak field [T]
!
!   XLELLIP=0.056   ! period length [m]
!   PERELLIP=31.    ! number of periods
!   ELLSHFT=0.25    ! shift of horiz. and vert. fields [periods]
!
!   NHARMELL=0 ! if not zero, magnetic field is calculated such, that
!   ! the energy of the NHARMth harmonic is HARMELL
!   ! signs and ratio of B0ELLIPV and B0ELLIP are kept
!   ! OVERWRITES PARKELL!
!
!   HARMELL=1000. ! photon energy of considered harmonic [eV]
!   ! -9999.: HARMELL=(FREQLow+FREQHIG)/2 or
!   ! or = FREQLow for single photon energy
!   ! other negative values: HARMELL denotes wavelength [nm]
!
!   XCENELL=0.0 ! longitudinal position of undulator center [m]
!   ELLTAP=0.0 ! scaling factor for taper
!
!$END
!
**-----+ Jed: wave.in (Text) 1353/3325 1 4:09pm -----

```

Fig. 4: The namelist ***ELLIPN*** of the elliptical undulator model

Here you could change the parameters of the selected elliptical undulator.

Note: The GUI *WAVESHOP* can't handle multiple occurrences of variables, i.e. it always takes the first value. So it's better to avoid it. If you want to keep a copy of an old value, change the line into a comment line.

Editing input files might appear a little old-fashioned today, but has the big advantage of easy documentation and allows to run and control the program from scripts, which can be very effective e.g in optimization procedures.



## 1. Example: Selecting a Magnetic Field Model

After

```
. $WAVE/shell/set_wave_environment.sh
```

our starting point is `$WAVE/input/wave.examples` . We copy it to `wave.in`:

```
cp $WAVE/input/wave.example wave.in
```

and enter

```
WAVESHOP
```

The GUI pops up, select in the upper menu bar *Comment* and to change the user title

```
WAVE.EXAMPLE
```

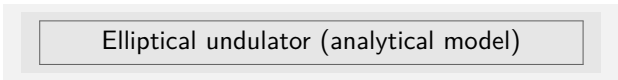
to

```
Planar Undulator
```

Hit *OK* to leave the widget. Select *Sel. Mag..*

A menu pops up:

– Click



Elliptical undulator (analytical model)

to deselect the current model and



Halbach undulator/wiggler with end poles

to select the planar undulator model.

Now we set up an undulator model with **99** main poles and two endpoles, i.e. a **50** periods undulator. The period length is **0.033m**. In addition, the third harmonic should be at **500eV** (Fig.5).

Note:

- Selecting a harmonic overrides the magnetic field value given in the shown menu.
- For this model,  $z$  is the longitudinal coordinate, while the WAVE coordinate system is such, that  $x$  is the longitudinale axis.

We close the last two menus and go back the main input menu.

Next, we adjust the observation pinhole.

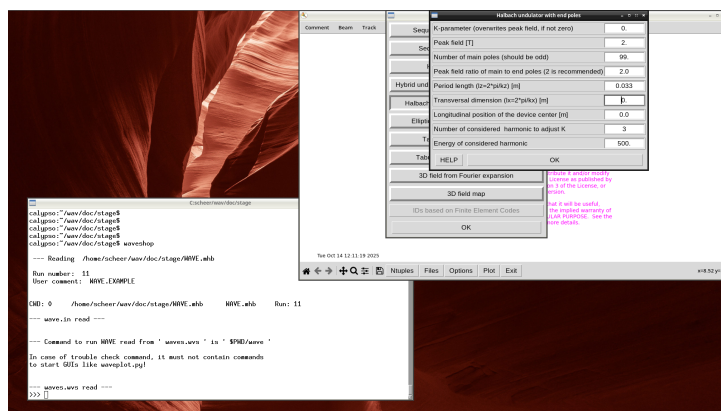


Fig. 5: Parameters of the undulator

- Click

Spectrum calculations

Pinhole definition

and set the pinhole width and height to **0.002**

- Hit **OK** and choose:

Spectral range

Set the lowest and highest photon energies to **495.0** and **505.0** eV, respectively.

We run WAVE by clicking **Run** in the toolbar.

In the terminal we see some warnings, which can be ignored. They could be relevant in the case of very precise calculations. To avoid them, one could increase the number of grid points in the Pinhole menu from 21 to 51, but that slows down everything for nearly no benefit.

Click the button **Plot**, select **Overview**, and check the results.

Before we finish this examples, we open the output file **wave.out** with a text editor.

This file contains a summary of the input variables and results of the calculations, as well as warnings and error messages. At the top you see your user comment (the variable **CODE** in **wave.in**) and the current run number, which is stored in the file **WAVE.CODE.DAT**. The output file is a good place to check the input parameters of the magnetic field model, the beam parameters etc..

As a beginner, you should at least once study the file thoroughly, although it might contain some information you don't understand at this moment...

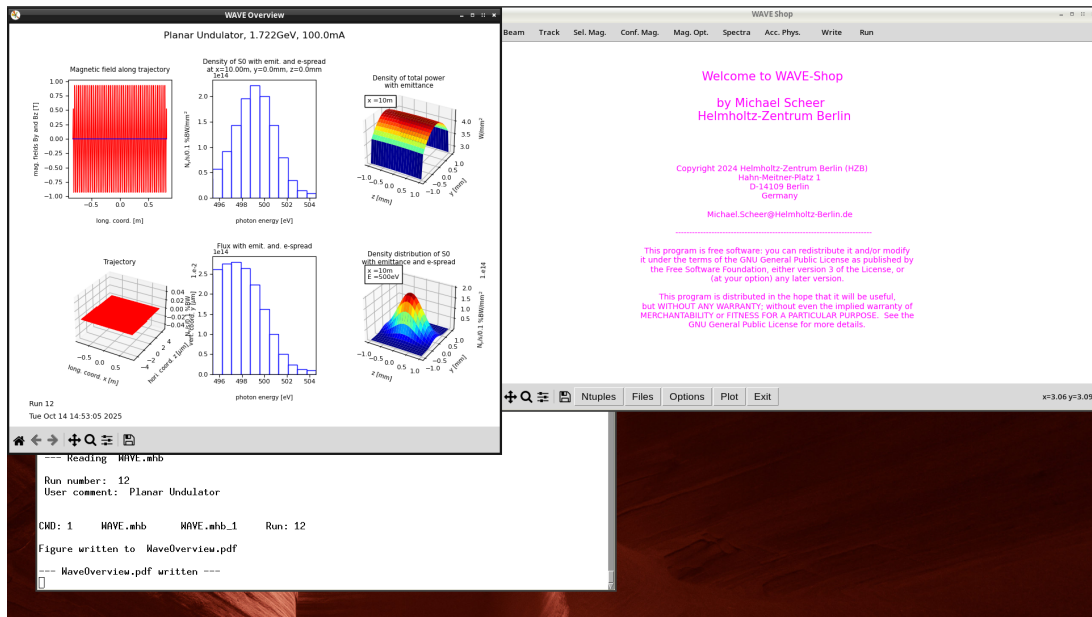
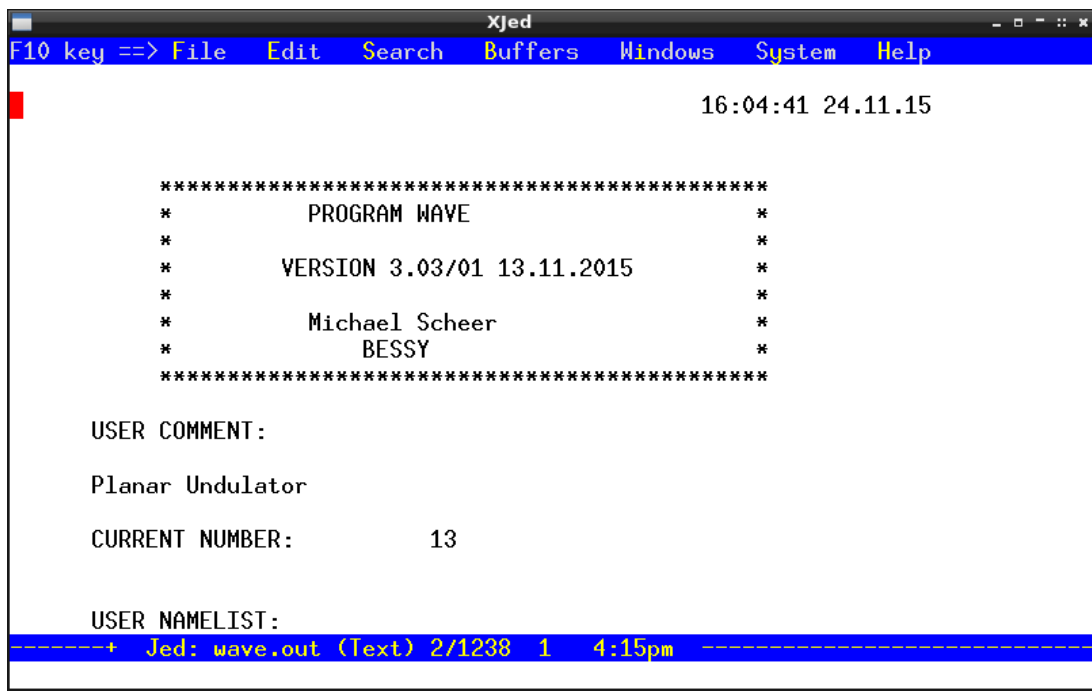


Fig. 6: Overview of plots

Fig. 7: The output file *wave.out*

Now it's up to you, to fumble around with the parameters of the menus described above and watch the effects. In case of trouble, you can always go back by copying *\$WAVE/input/wave\_example\_1.in* to *wave.in*.

Proposed exercise: Compare *wave.in* and *\$WAVE/input/wave.examples*.

## 2. Example: Tracking through Magnetic Fields

This examples illustrates the tracking mechanism of WAVE (and by the way the use of the command line... for the plotting.) After

```
. $WAVE/shell/set_wave_environment.sh
```

We start from `$WAVE/input/wave.examples` and use the python script `../python/wave_example_2.py` to create `wave.in`. But before, we have a look on it. We open `../python/wave_example_2.py` with an text editor (Fig. 8):

```
ReplList = [ \
["CODE='WAVE.EXAMPLE'", "CODE='Example 2: Tracking through Magnetic Fields'"], \
["BYGOFF=0.0", "BYGOFF=0.00005"], \
["KELLIP=1", "KELLIP=0"], \
["KHALBASY=0", "KHALBASY=1"], \
["B0HALBASY=2.", "B0HALBASY=1."], \
["PKHALBASY=1", "PKHALBASY=0"], \
["AHPOL=21.", "AHPOL=99."], \
["ISPEC=1", "ISPEC=0"], \
]
```

Fig. 8: First part of the script to set-up `wave.in` for the second example `wave.out`

The list `ReplList` contains the strings to be replaced in `wave.in`. Each line contains two strings. The first string in `wave.in` is replaced by the second. Most of the variables you known from example 1, but the variable `ISPEC` belonging to the namelist `$CONTRL` is new. Changing `ISPEC=1` to `ISPEC=0`, switches the spectrum calculations off. Also new is the variable `BYGOFF`, which is a global field offset added to the undulator field, e.g. to simulate the earth field.

Now we execute the script, copy the resulting file, and run WAVE :

```
python3 ../python/wave_example_2.py
cp ../input/wave_example_2.in wave.in
wave
```

On the terminal, two warnings appear, indicating that the trajectory is not correctly adjusted. This means, a final kick and a final offset remain. We start the plotting GUI:

```
wplot
```

Click the button **Plot**, select **Overview**, and check the results.

We see in the overview Fig. 9 that the trajectory is distorted as expected due to the external field,

Sometimes, it's useful to adjust the trajectory, e.g. if you want to have a symmetric one with respect to the center of the straight section or if for a given field model the correct endpole constellation is missing. This can be achieved with by tuning the electron parameters in the namelist `$CONTRL`:

The most interesting variable is `XINTER`, which defines the x-coordinate the other variables like `YSTART` or `VZIN` refer to.

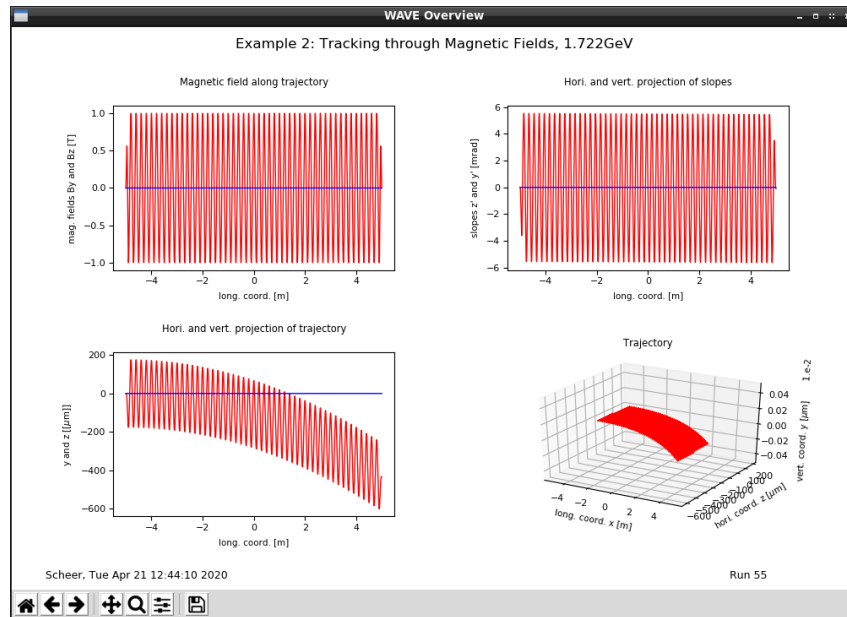


Fig. 9: The trajectory with  $XINTER=-9999$ .

Edit *wave.in* and set

$XINTER=0$ .

Then we save the file and enter:

```
wave
wplot
nz()
```

and see (Fig. 11) a trajectory, where the electron passes the origin with zero slope according to:

```
YSTART=0.
ZSTART=0.
VXIN=1.
VYIN=0.
VZIN=0.
```

at

$XINTER=0$ .

Thus, if  $XINTER$  is not  $-9999$ , WAVE tracks first the electron from  $X = XINTER$  back to  $XSTART$  and then starts the actual tracking through the magnetic field.

You can save the picture with the corresponding toolbar button or typing "s" when the cursor is over the plotting window or by typing e.g.

```
pplot("trajectory.png")
```

or

```
pplot("trajectory.pdf")
```

```

Xjed
F10 key ==> File Edit Search Buffers Windows System Help
XSTART=9999. ! initial x of the trajectory [m]
! 9999. means default is used if possible
! i.e. entrance of the device
! -9999. XSTART is set to -XSTOP
XINTER=-9999. ! if XINTER not -9999. and not missing tracking starts
! at XINTER. The particle is tracked forward or
! backward to XSTART.
! XSTART,YSTART,...,VXIN,... are overwritten
! for restart of tracking now beginning at XSTART
! 9999. means default is used if possible
! i.e. entrance of the device
! XINTER=XSTART=9999. is allowed but makes no sense
XSTOP=9999. ! final x of the trajectory [m]
! 9999. means default is used if possible
! i.e. exit of the device
YSTART=0.0 ! initial y of the trajectory [m]
ZSTART=0.0 ! initial z of the trajectory [m]
VXIN=1.0 ! initial x-component of velocity vector (relative)
VYIN=0.0 ! initial y-component of velocity vector (relative)
VZIN=0.0 ! initial z-component of velocity vector (relative)
-----+(Jed 0.99.19U) EDT: wave.in.txt (Text) 201/3175 Advance 11:28am

```

Fig. 10: Electron variables

in the Python shell. (If the overview is plotted a file *WaveOverview.pdf* is written automatically.) The last plotting command

*nz()*

can of course also be triggered from the GUI menu. The "n" refers to the fact, that the data stem from an n-tuple. You can also try

*nby()*

and so on. You might have already noticed, that the plotting commands executed by the GUI are echoed in the Python shell. The command echoes show you the syntax and parameters of the plotting commands applied. So you can learn short-cuts for plotting from the Python shell.

If you don't see these command echos, the configuration file *waveplot.cfg* is missing in your working directory. You find an example in the input-directory *\$WAVE/input*.

If *waveplot.cfg* is present, plotting options are read from this file. E.g. you can set the window size, trigger saving of all plots or writing plotted data to ASCII-Files. To see all options, check this file.

Some options can also be toggled in the GUI or by commands, e.g.

*optecho(1)*

or

*optecho(0)*

or

*optstat(1)*

or

*optstat(0).*

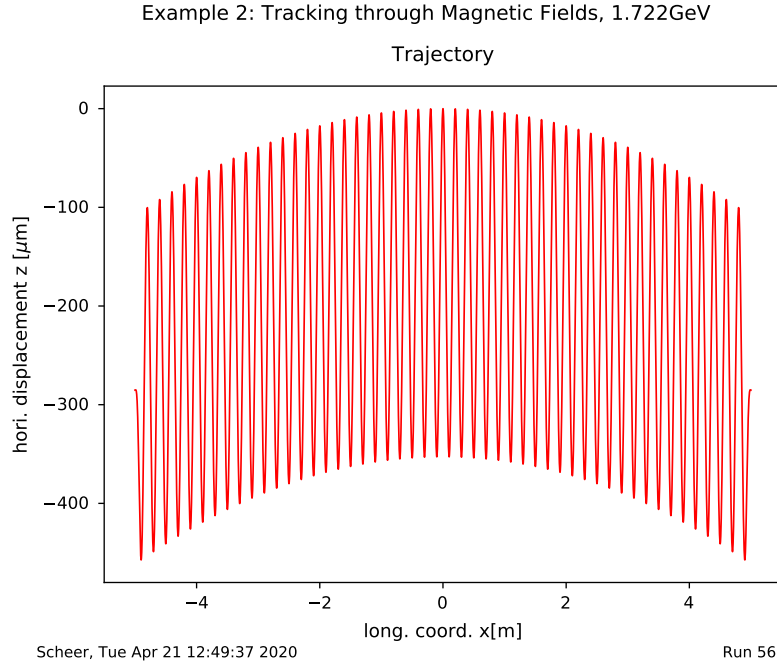


Fig. 11: Adjusted trajectory with *XINTER=0.* .

Note: If you calculate synchrotron radiation from this trajectory, i.e. *ISPEC=1* , the back tracking at the beginning does, of course, not contribute to the radiation or irradiated power of the device. By the way, this power is given in the output file *wave.out* . The power is normalized to the beam current *DMYCURR* in the namelist *\$CONTRL* . In *wave.out* you also find the energy loss of the electron having passed the magnetic field. This is controlled by *IENELOSS* also namelist *\$CONTRL* . For very long undulators or wigglers, the energy loss influences the trajectory and the spectrum of synchrotron radiation. For a single, typical insertion devices, this can be neglected, i.e. *IENELOSS=0* can be used.

### 3. Example: The Concept of Source Points

Synchrotron radiation is mainly emitted into a narrow cone on the scale of  $1/\gamma$ . Consider a wave-length shifter (WLS) with a main pole and two endpoles. Such a device has a deflection angle that is usually much larger than  $1/\gamma$ . Therefore, an observer at point  $P$  sees up to three sources of radiation from the WLS, namely the points of the trajectory, where the electron points to  $P$  as sketched in Fig. 12. These points, we call source points. Actually, they are not really points, but pieces of the trajectory due to the finite opening angle of the radiation cone.

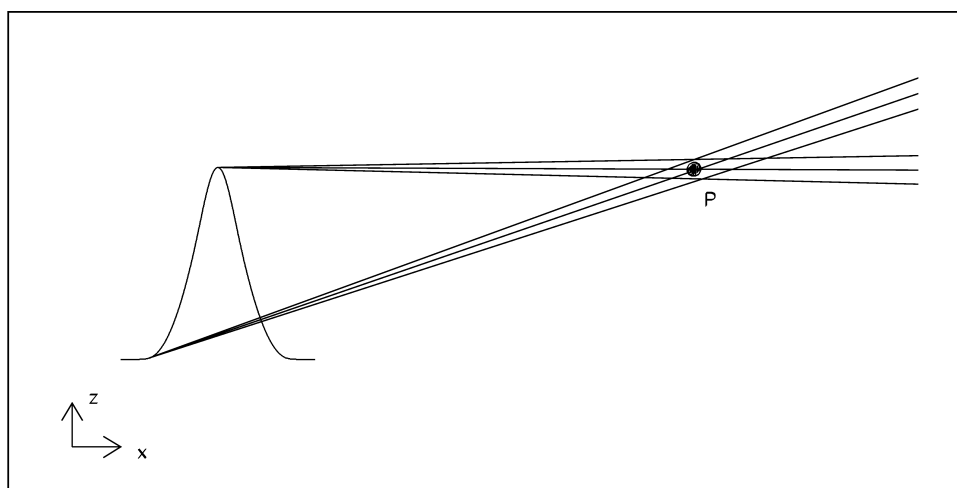


Fig. 12: Source points of a wave-length shifter for observation point  $P$

Let's have a closer look. Enter in the terminal:

```
. $WAVE/shell/set_wave_environment.sh
cp ../input/wave.example wave.in
wvshop
```

- The GUI pops up, select in the upper menu bar **Comment** and change the user title

User title

Example 3: The Concept of Source Points

- Click **Track** in the toolbar  
and set

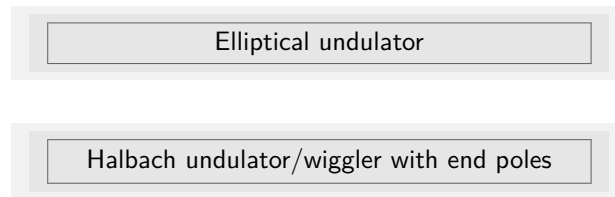
number of steps per meter

100000

to increase the precision of the spectrum calculations in the undulator mode. Hit **OK**.

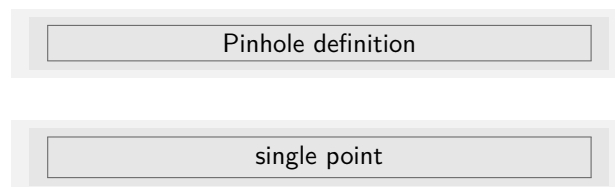
- Click **Sel. Mag.** in the toolbar and in the menu





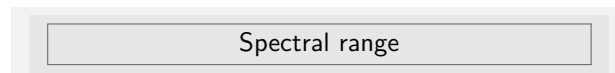
and set the peak field to **6.0T** , the number of main poles to **1** , the period length **0.7m** . Hit **OK**, again **OK**.

- Click **Spectra** in the toolbar and in the menu



Click **OK**.

- Click



and set the lowest photon energy to **50.** , the highest to **25000.** , and the number of photon energies to **2001** , then hit **OK** (Fig. 13).

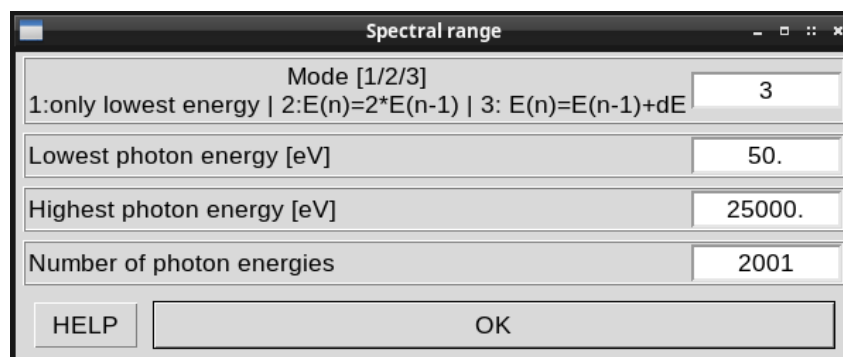


Fig. 13: Setting of the spectral range

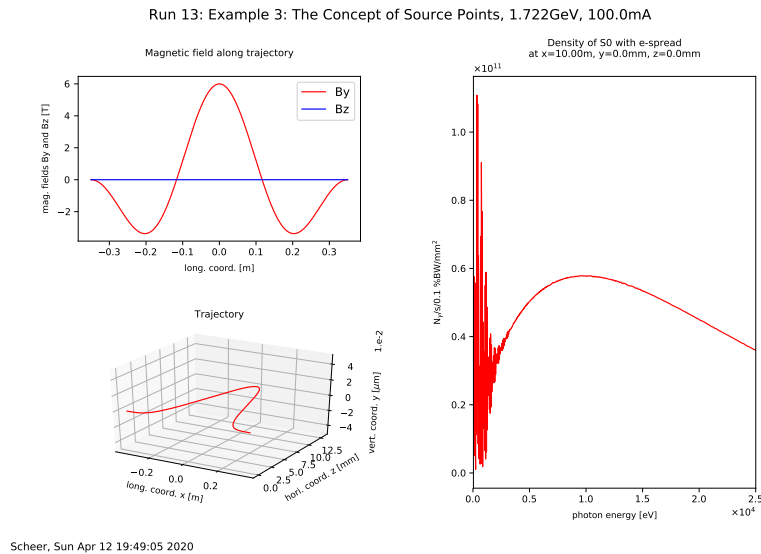


Fig. 14: The spectrum of an WLS calculated in the undulator mode

Close the menu and click **Run** in the toolbar and plot the overview.

The spectrum (right plot in the overview Fig. 14) looks a little weird. We leave the GUI, edit **wave.in** and set

```
FREQLOW=2000.
FREQHIG=2010.
NINTFREQ=201
IEFOLD=3
```

To run WAVE , enter:

**wave**

When WAVE has finished, enter

**wplot**

We choose in the plotting GUI

**Plot/Spectra/Flux-density/Fd**

The spectrum (red) in Fig. 15 is not as smooth as we would expect from a wave-length shifter. The synchrotron radiation of a single electron is fully coherent, thus there are interferences between the radiation of the different poles of the WLS. But the phase advance between the poles is so high, that the corresponding huge number of oscillations can not be resolved by the integration algorithm of WAVE . The observed spectrum is therefore a mixture of real interferences and a numerical noise depending on the integration parameters.

However, the energy-spread of a real beam smears out the spectrum. To plot the smeared spectrum goto to the plot GUI and select:

*Options/Same Zone*  
*Options/Next Color*  
*Menu/Spectra/Flux-density/Fd with e-spread*

The resulting spectrum (blue line) is smooth as would be expected from a wave-length shifter.

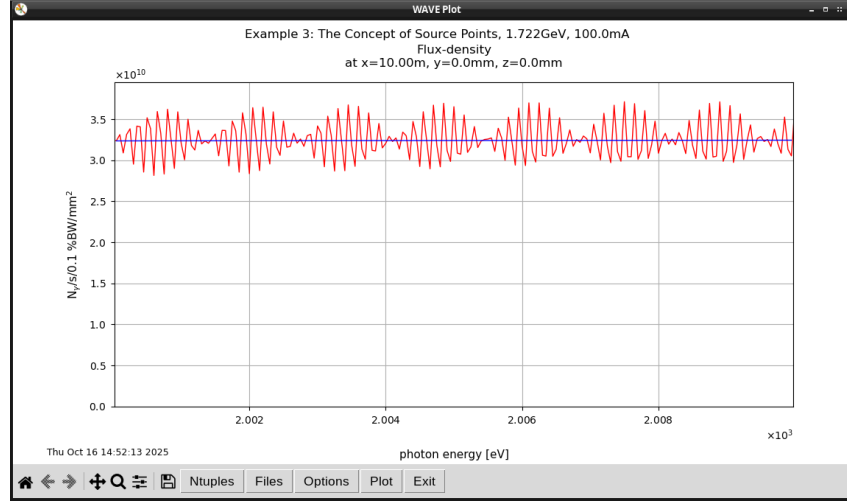
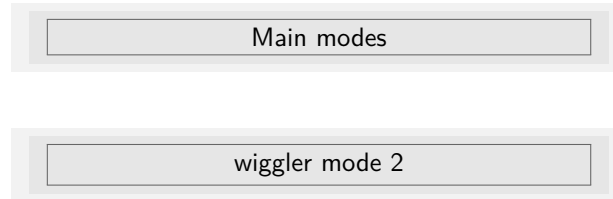


Fig. 15: The spectrum of an WLS calculated in the undulator mode.  
 Red: Without e-spread, blue: with e-spread.

Terminate the plotting and start the GUI *WAVESHOP*.

In the toolbar hit *Spectra* and select in the menu "Spectrum calculations"



Hit *OK* and run WAVE .

To plot the spectrum including the energy-spread, we choose in the plotting GUI

*Menu/Spectra/Flux-density/Fd with e-spread*

or enter in the Python shell

`hcfluxden("fde")`

Fig. 16 shows the same result, as in the previous run with the undulator mode. We conclude, that the interferences between the poles of the WLS are real, but can not be calculated correctly and can not be observed for real beams with a finite energy-spread. Hence, it is justified to neglect them,

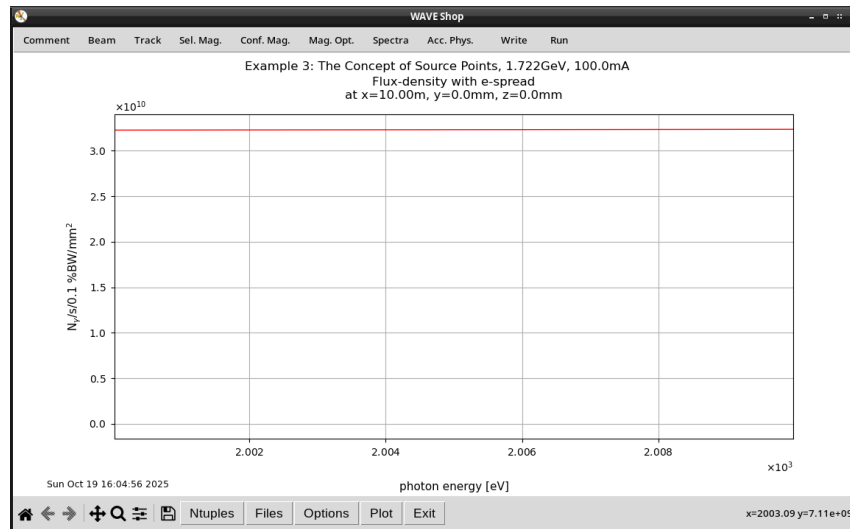


Fig. 16: The spectrum of a WLS calculated in the wiggler mode including the energy-spread of the beam

i.e. to calculate the radiation from each pole separately and to sum up the contribution of all poles incoherently. This is what the wiggler mode of WAVE does. The decision, if the wiggler mode should be applied (and especially together with Schwinger's formula) depends on the regime, where the spectrum is calculated. For a wiggler, this means the undulator mode is needed in the spectral range of the lower harmonics. In doubt, try the different modes and have a look on the results.

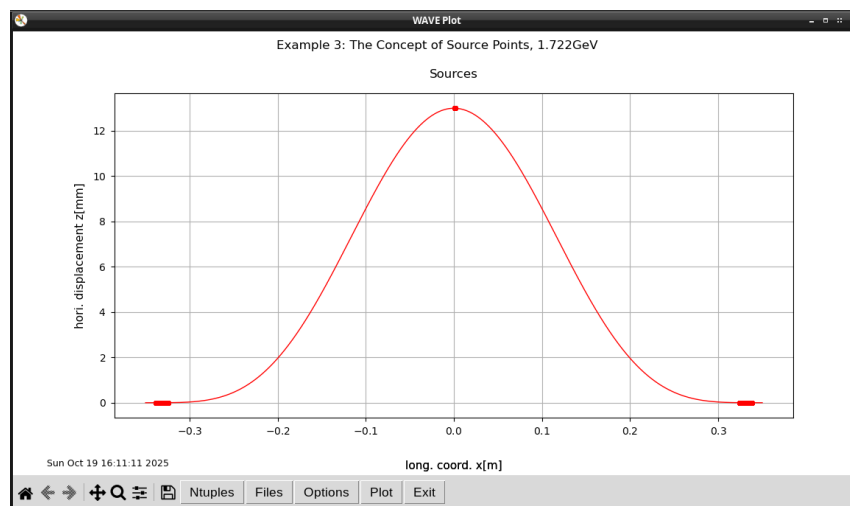


Fig. 17: Source points of a WLS

In the plotting GUI, choose:

*Menu / Fields and Trajectory / Sources / z(x)*

As shown in Fig. 17, WAVE has found three source points. How does it work? WAVE tracks first the electron and stores the trajectory. Then it checks for all steps of the trajectory, if a beam with

a given opening angle passes at least partially a collimator. The first step satisfying this, is marked as the beginning of a source, the last as the end.

This procedure and the set-up of the collimator are controlled by the variables of the namelist *\$COLLIN*. The main modes of WAVE namely the undulator and wiggler modes set default values for these variables, which should work in most cases. In addition, the main modes control the integration method *ISPECMODE* and the number of integration steps *NLPOI* used to calculate the spectrum.

Proposed exercise:

Try the expert mode, i.e. *IUNDULATOR=0*, *IWIGGLER=0* and play with the variables *WGWINFC*, *NLPOI* and the variables of the namelist *\$COLLIN*.

#### 4. Example: APPLE-II Undulator

The most realistic undulator model <sup>1</sup> of WAVE is an undulator build of permanent magnet blocks (rare earth compounds like NdFeB, shortly REC). The field of these blocks is calculated with the current sheet method. Due to the large number of blocks of an undulator the field calculations are rather slow, but the field is very realistic, and an undulator can be modelled with all its features like gap and shift changes.

The namelist *\$REC* holds the variables of the undulator. It also includes gap taper and field errors. To set up the input file *wave.in*, enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_4.py
cp $WAVE/input/wave_example_4.in wave.in
```

After the WAVE run, you'll find a file named *rec\_plotm.eps* , which shows the set-up of the undulator (Fig. 18).

20.10.2025 11:30

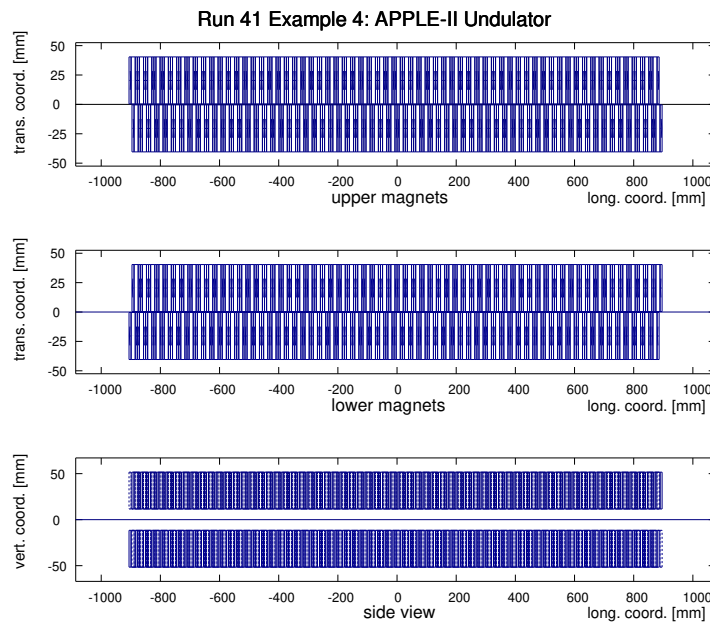
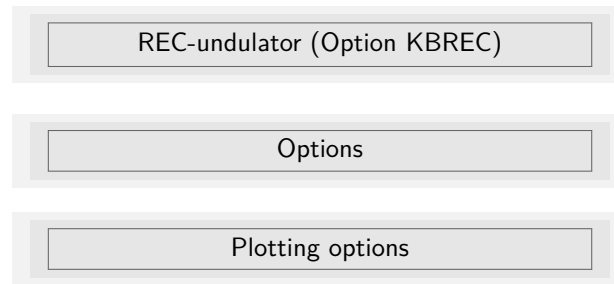


Fig. 18: REC model of an undulator

The figure shows the upper and lower rows of magnets as top views and a side view of both. To see details we click **Conf. Mag.** in the toolbar of *WAVESHOP* and

<sup>1</sup>See also example ?? for an even better model by interfacing to UNDUMAG. UNDUMAG allows to take into account the interaction between the neighbouring magnets, while WAVE only considers static magnets with  $\mu = 1$ . This interaction might affect especially the first and second integrals, i.e. the kick and off set at the end of the trajectory.



and set the plotting range

RPLXMN = -920.

RPLXMX = -850.

and rerun WAVE .

20.10.2025 11:21

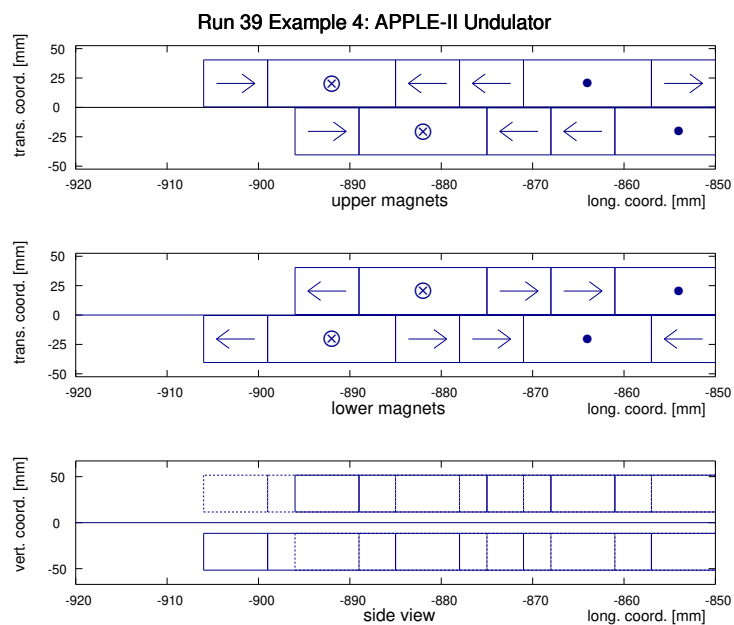


Fig. 19: Endpoles of the REC model

Now, you see the details of the endpoles, the shift, and the magnetization of the blocks. More complex situation can be handle by an input file containing arrays of magnet. This will be explained in one of the following examples.

The overview of this run is shown in Fig. 20. It shows the elliptical trajectory and the corresponding first harmonic.

Example 4: APPLE-II Undulator, 1.722GeV, 100.0mA

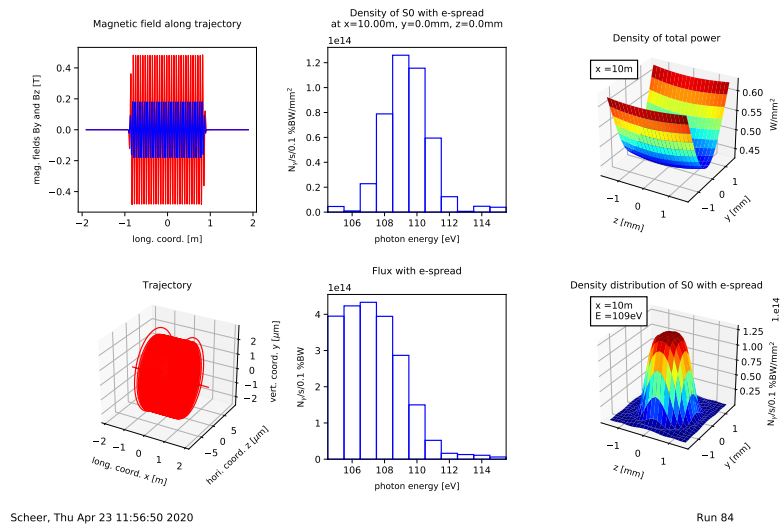


Fig. 20: Example of an APPLE-II undulator

Proposed exercise:

In the option menu of the REC-Undulators is a menu to define errors of the magnetic blocks. Give it a try, and check also the magnetic field errors option in the menu of the global magnetic field options.



## 5. Example: Simple Build-In Optimization

A very common situation is, that you want to tune input parameters with respect to a figure of merit. WAVE has build in two simple procedures to accomplish that. The first is an iterativ estimation for the case of one parameter to adjust, the second a Monte-Carlo for more than one parameter.

Consider example 4. For the given gap, we want to find the shift such, that the vertical and horizontal field of the undulator are the same, i.e. the device acts as an circular undulator.

In a terminal, we enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_5.py
cp ../input/wave_example_5.in wave.in
wave
```

We see in **wave.out** , that the max. vertical and horizontal fields **BYMAX** and **BZMAX** differ. Now, we adjust the shift parameter of the undulator. Enter:

```
cp ../input/wave_example_5.adjust wave.adjust
cp ../python/wave_adjust_example_5.py wave_adjust.py
```

Have a look on **wave.adjust** (Fig.22). When a file like this is passed as an argument to WAVE, WAVE will adjust the parameter **URSHIFT(1)** of **wave.in** by a Monte-Carlo search. Of course, this also works with other filenames and variables of **wave.in**. During the optimization procedure WAVE executes the script **wave\_adjust.sh** , which calls another instance of WAVE and **wave\_adjust.py**.

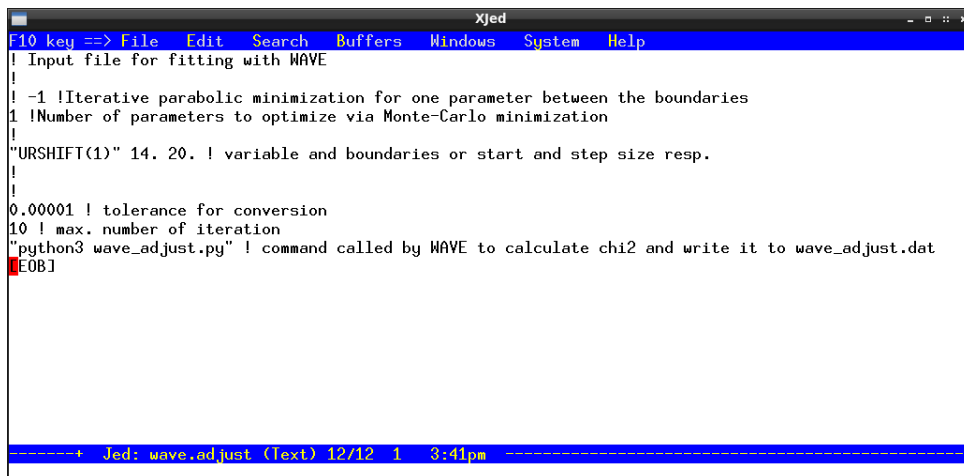
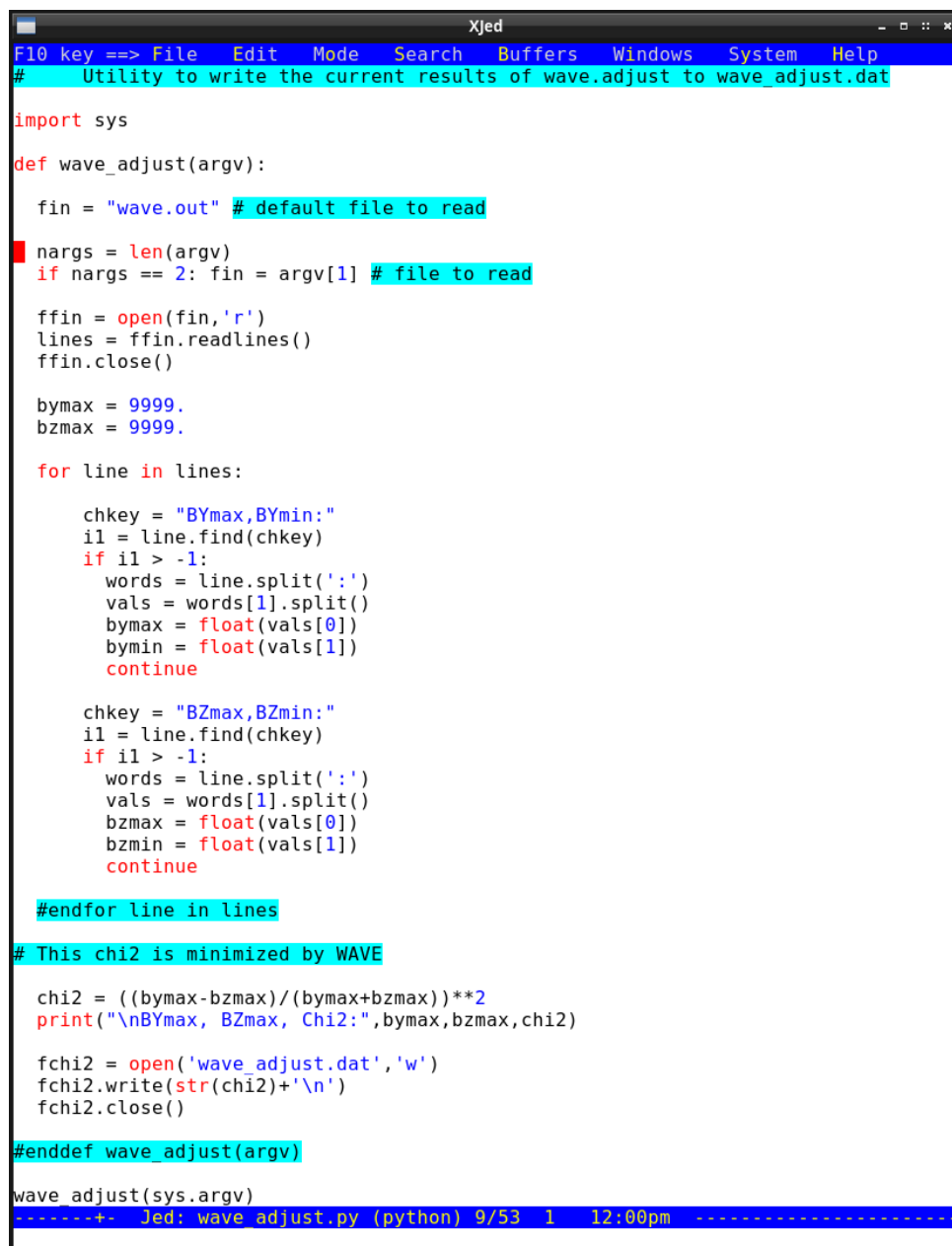


Fig. 21: Input file for the optimization procedure

After the run of this instance, the command given in the last line of **wave.adjust** is executed by the master process. In our example, this is the python script **wave\_adjust.py** . What does this script

(Fig.22) do?



```

Xjed
F10 key ==> File Edit Mode Search Buffers Windows System Help
# Utility to write the current results of wave.adjust to wave_adjust.dat

import sys

def wave_adjust(argv):

    fin = "wave.out" # default file to read

    nargs = len(argv)
    if nargs == 2: fin = argv[1] # file to read

    ffin = open(fin,'r')
    lines = ffin.readlines()
    ffin.close()

    bymax = 9999.
    bzmax = 9999.

    for line in lines:

        chkey = "BYmax,BYmin:"
        il = line.find(chkey)
        if il > -1:
            words = line.split(':')
            vals = words[1].split()
            bymax = float(vals[0])
            bymin = float(vals[1])
            continue

        chkey = "BZmax,BZmin:"
        il = line.find(chkey)
        if il > -1:
            words = line.split(':')
            vals = words[1].split()
            bzmax = float(vals[0])
            bzmin = float(vals[1])
            continue

    #endfor line in lines

# This chi2 is minimized by WAVE

    chi2 = ((bymax-bzmax)/(bymax+bzmax))**2
    print("\nBYmax, BZmax, Chi2:",bymax,bzmax,chi2)

    fchi2 = open('wave_adjust.dat','w')
    fchi2.write(str(chi2)+'\n')
    fchi2.close()

#enddef wave_adjust(argv)

wave_adjust(sys.argv)

-----+-- Jed: wave_adjust.py (python) 9/53 1 12:00pm -----

```

Fig. 22: Example of a program to calculate the figure of merit

It opens *wave.out* , and looks for the lines containing the keywords like *BYmin*. When a matching line is found, the program reads from the line the values of *By* or *Bz*, respectively, and calculates the  $\chi^2$  to be minimized. The resulting  $\chi^2$  is written to *wave\_adjust.dat* and evaluated by the master process.

Let's try. Enter:

```
cp wave.in wave_adjust.in
wave wave.adjust
```

WAVE loops and writes **wave\_adjust.out** . The data in this file show a minimum for the shift of about **19mm** (Fig.23). This minimum value found is also used to re-run WAVE.

Start the plotting GUI by entering:

```
wplot
```

and enter in the python shell:

```
adj = ncread("adj","sh:chi2","wave_adjust.out")
nplot(adj,"sh:chi2")
txyz("wave_adjust.out","Shift[mm]","Chi2")
```

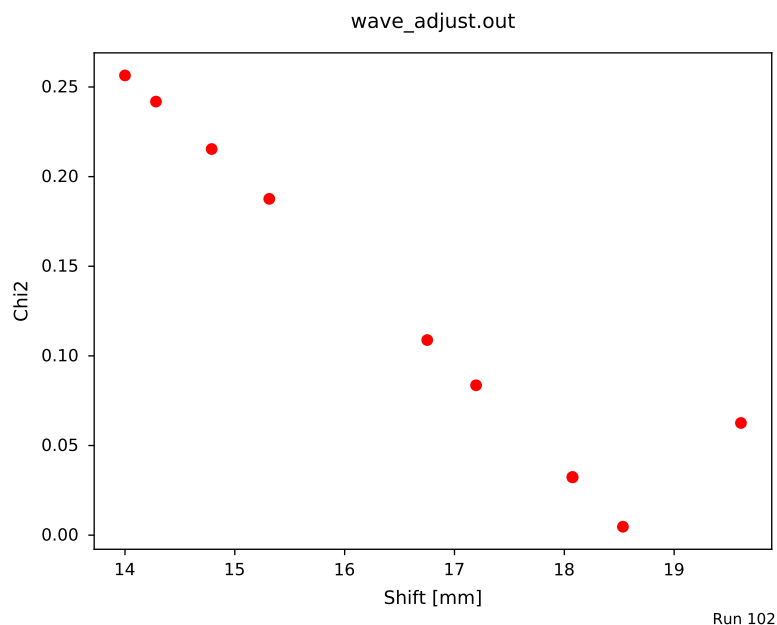


Fig. 23: Random variation of URSHIFT(1)

The plot you get, might differ from that shown here, due to different seeds of the Monte-Carlo search.

To get a precise value, we edit **wave.adjust** , and set the number of parameters to **-1** and the boundaries to **18.0** and **20.0** . Then we repeat

```
cp wave.in wave_adjust.in
wave wave.adjust
```

and find now in **wave.in** the best value of about

*URSHIFT(1)= 18.6*

Since we have required a precision for the minimum  $\chi^2 \leq 10^{-6}$ , the ratio of the vertical and horizontal field now agrees on the scale of  $\sqrt{10^{-6}}$ .

The effort in this example might appear quite high for such a simple problem, which could be solved sufficiently by a quick interpolation. However, it shows the usage of scripts and the advantage of command line driven optimization procedures.

## 6. Example: "Real" Beams, Part I

Up to now, we have only considered single electron beams, except in example 3, where we have seen the effect of the energy-spread of the beam. In this example, we learn about the first of two methods how WAVE takes the effects of real beams, i.e. emittance and energy-spread into account.

In modern low emittance storage rings the spatial and spectral distribution of the synchrotron radiation does not vary very much for the different electrons in the beam phase-space. Thus, the shape of the radiation cone is more or less the same, it is only shifted according to the position and slope of the considered electron. Therefore, it is possible to take the beam emittance into account by folding the radiation cone of the reference electron with a distribution that reflects the phase-space. This distribution is usually a 2D-Gaussian with

$$\begin{aligned}\Sigma_y^2 &= \sigma_y^2 + d^2 \times \sigma_y'^2 \\ \Sigma_z^2 &= \sigma_x^2 + d^2 \times \sigma_x'^2\end{aligned}$$

where  $\sigma_y, \sigma_y', \sigma_z, \sigma_z'$  denote the vertical and horizontal sizes and divergences of the beam and  $d$  the distance between the source and the observation plane. Enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_6.py
cp ../input/wave_example_6.in wave.in
wave
wplot
```

Use the plotting GUI to produce the Fig. 24 or enter in the python shell:

```
zone(1,2,1)
ndistpinh()
nextzone()
ndistpinv()
```

If you want to change parameters for various runs, but plot always the same, you can put these lines into a file

**waveplot\_startup.py**

The commands in this file are executed at the start of **wplot**, until the end of the file is reached, or a line with a key-word **EOF** is found. Of course, this can also handle the plotting zones by using the **Options** menu of **wplot**.

The mean value of the histograms agree with the position of the observation pinhole as given in **wave.out**. We see, that the position of the pinhole is shifted. This has been achieved by setting

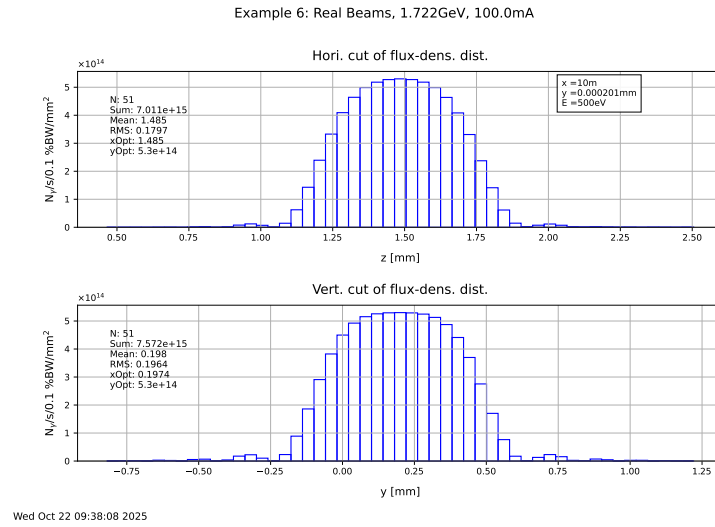


Fig. 24: Horizontal and vertical cuts through the flux-density distribution

```
YSTART=56.1e-6
ZSTART=648.e-6
VYIN=12.87e-6
VZIN=74.4e-6
PINCEN(2)=-9999.
PINCEN(3)=-9999.
```

in *wave.in*. The values for the start conditons of the electron correspond to three sigmas of the beam sizes and divergences as given in the namelist *\$WFOLDN*. The values for the center of the pinhole trigger the shift of the pinhole, as is explained in the namelist *\$PINHOLE*.

When we set in *wave.in*

```
YSTART=0.0
YSTART=0.0
ZSTART=0.0
VYIN=0.0
VZIN=0.0
```

as usual, we get nearly the same histograms, except that they centered on-axis. Hence, the ansatz of the folding procedure is verified.

A similar folding procedure is applied for the spectrum to take into account the energy-spread of the beam (*ESPREAD* in the namelist *\$WFOLDN*). This procedure makes use of the fact, that the harmonics of a undulator depend on the square of the beam energy  $E_{beam}$ . Thus, the shift of the spectrum is in first order given by

$$\frac{\Delta E_{phot}}{E_{phot}} = 2 \times \frac{\Delta E_{beam}}{E_{beam}}$$

while the shape stays in a good approximation the same.

Proposed exercise:

Try to verify the last statement by using WAVE .

## 7. Example: Global Magnetic Field Options

WAVE offers in the namelist `$B0SCBLOBN` some options, e.g. shifting, scaling, and symmetry operations of the magnetic field. It also offers to repeat the set-up periodically. In this example, we make use of these options in order to calculate the spontaneous synchrotron radiation of a very long undulator section including phase shifters. But first, we consider a single cell of this section (Fig. 25). We enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_7.py
cp $WAVE/input/wave_example_7.in wave.in
wave
wplot
```

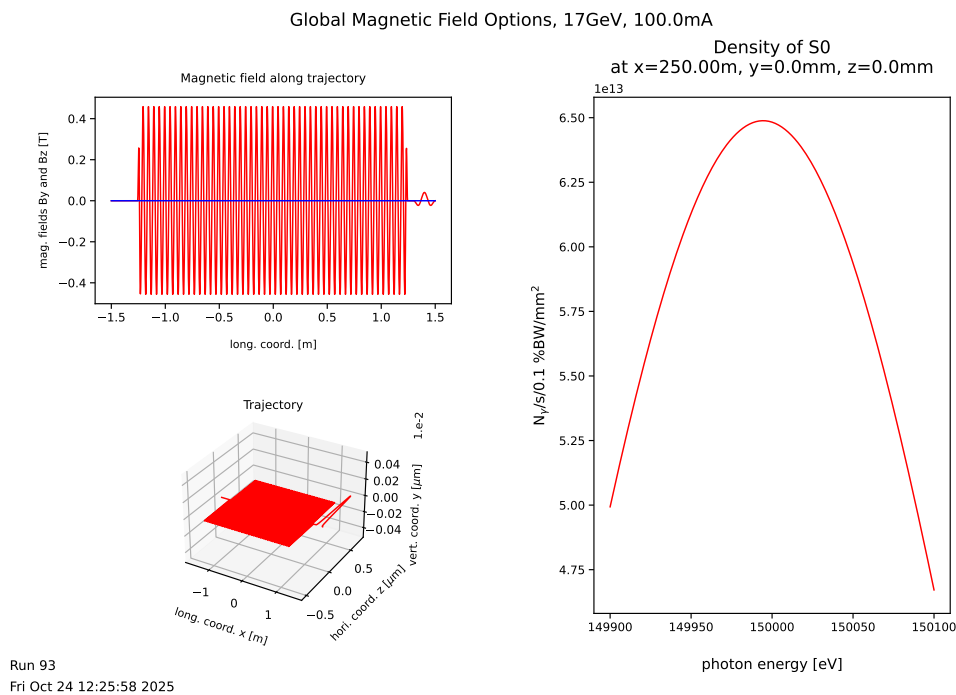


Fig. 25: Field and 9th harmonic of a single undulator section

As the overview shows, the field is a superposition a planar undulator (namelist `$ELLIPN`) and a phase shifter (namelist `$HALBASY`). If `IBSUPER=1` in `wave.in`, WAVE accepts several field selections. The undulator parameters are set such, that the 9th harmonic is at **150 keV** for a beam energy of **17 GeV**. We now make use of the periodic field option. Have a look on the corresponding variables in the namelist `$B0SCGLOBN`:

```
IPERIODG=1
PERIODG=3.
PEROFFG=-1.5
```



This means, that the magnetic field is periodically repeated starting from  $X=-1.5$  with a period-length of 3 meters.

We set in *wave.in*:

`XSTOP=88.5`

`IHTRSM=10`

and run WAVE and get the results shown in Fig. 26

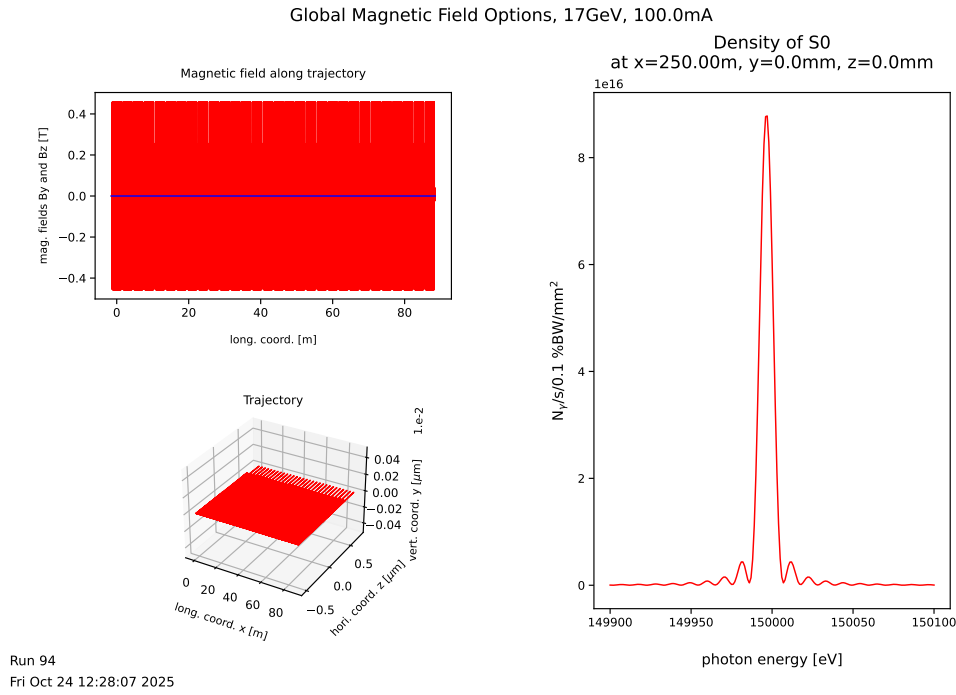


Fig. 26: Field and 9th harmonic of a long undulator section

Passing a long undulator section an electron beam loses a significant amount of energy. Hence the K-parameter of the down-stream undulators does not match the beam energy anymore. This results in a broadening of the spectrum. This can be simulated by setting

`IENELOSS=1`

in *wave.in*. We run WAVE and get the results in Fig. 27.

To compensate it, we must adjust the field of each undulator accordingly. This can be achieved with the field option

`JBMASK=100`

and by providing a file *wave.bmask*, which contains a number of  $X$ -Intervalls and a scaling factor for the magnetic field for each intervall. The last *wave.out* gives the total and relative energy loss of the beam due to the 30 undulators. From this number we calculate the energy loss of a single undulator cell, and use the relation

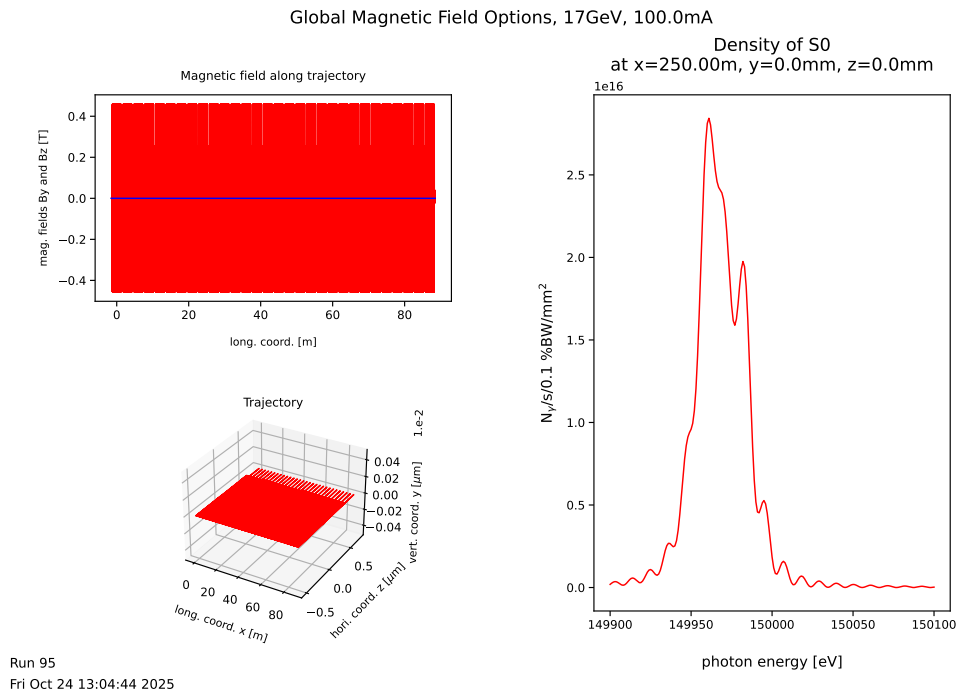


Fig. 27: Field and 9th harmonic of a long undulator section with continous energy loss of the beam

$$\frac{E_{beam}^2}{1 + K^2/2} = const$$

to calculate the correction factor for the undulator field, as it is done in the function **taper()** , which also writes the file **wave.bmask**

We use this function:

```
wplot
taper()
Quit()
```

The default values of this function correspond to this example. Beside the relative energy loss the starting  $K$ -value is taken from **wave.out** of the previous run as well (Look for "deflection parameter").

In **wave.in** we set

```
JBMASK=100
```

Please, also read the comments of the corresponding namelist **B0SCGLOBN** carefully and notice the difference between **JBMASK** and **IBMASK**.

We run **WAVE** again and see in Fig. 28 that the harmonic of the undulator recovers.

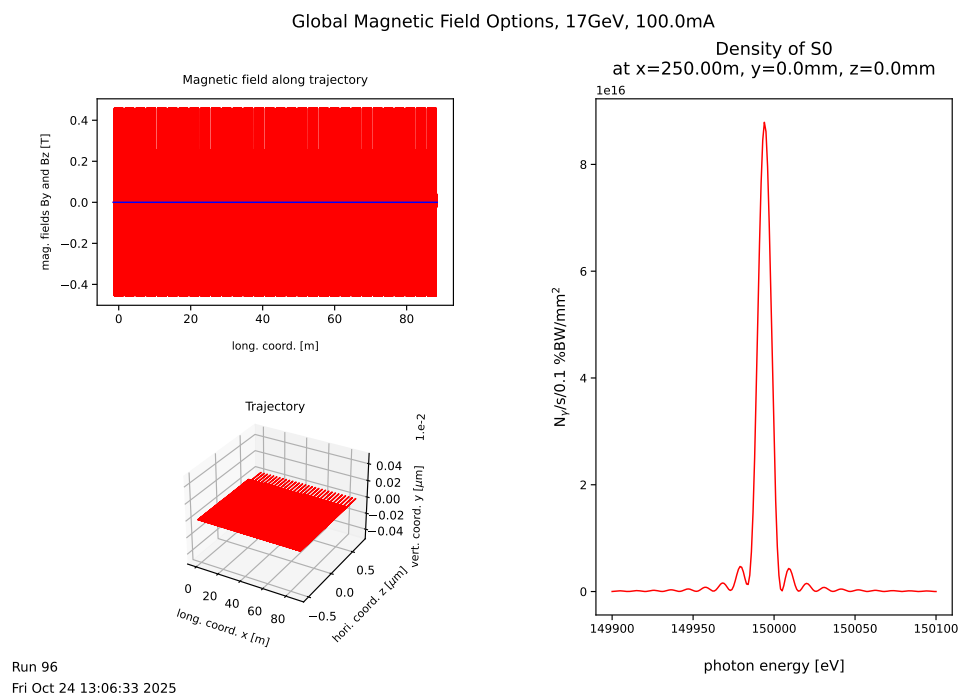


Fig. 28: Field and 9th harmonic of a long undulator section with continous energy loss of the beam and corrected undulators

## 8. Example: Magnetic Field Errors

The quality of an undulator depends essentially on the errors of the magnetic field. Field errors not only effect the trajectory and hence e.g. the orbit of electrons in storage rings, but also reduce the spectral brightness of the device, in particular for the higher harmonics.

WAVE offers some options to study these effects. We enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_8.py
cp $WAVE/input/wave_example_8.in wave.in
wave
wplot
```

We look at the spectrum of the flux-density. Click:

*Plot / Spectra / Flux-density / Fd*

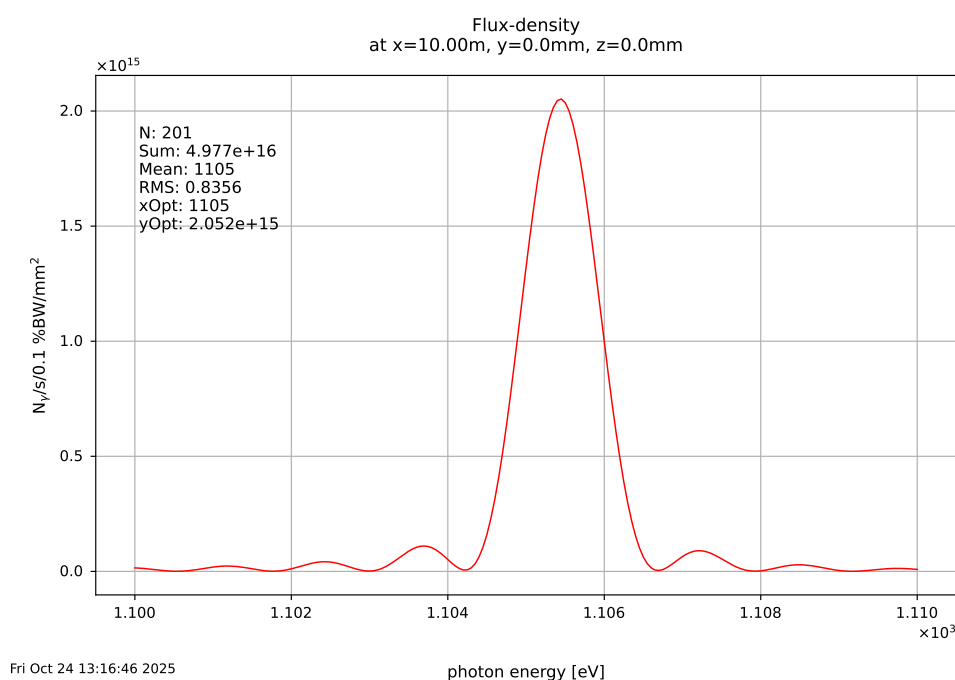


Fig. 29: Ninth harmonic of a planar undulator

The main errors of permanent magnet undulators arise from fluctuations of the magnetization of the block. This can be a variation in the strength or the direction of the blocks. In *wave.in* we set

```
BCRAN=0.001
UBANGERR(1)=0.2
```

In addition, feel free to modify the start of the random generator by changing *IRNMODE* in the namelist *\$RANDOMN*. Of course, then you will reproduce the plots of this example. . .

Then we run WAVE again and look at the trajectory (Fig. 30) and the spectrum by plotting the overview.

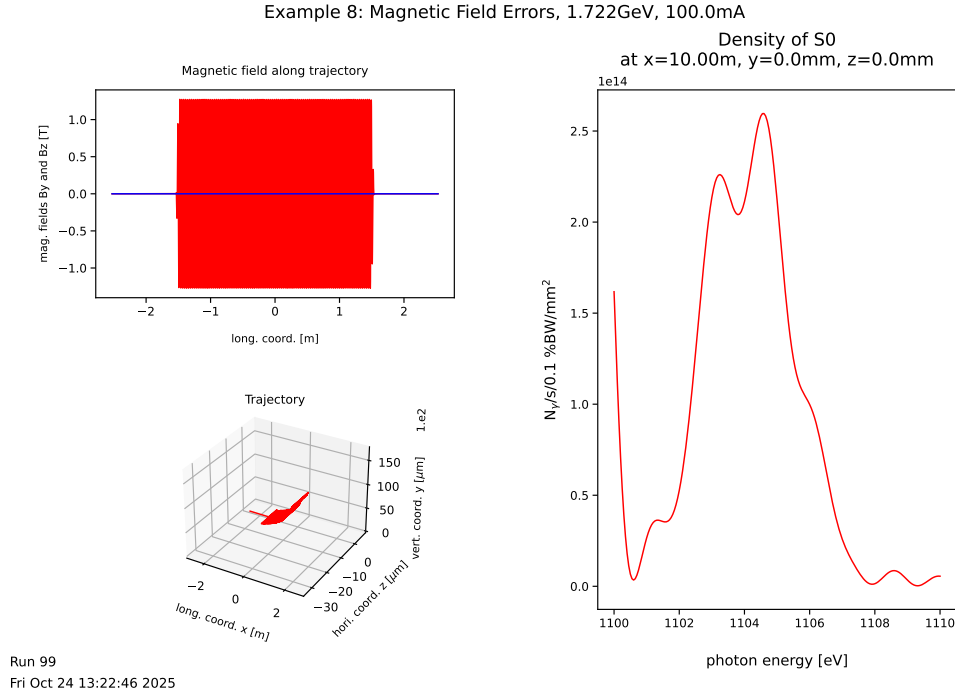


Fig. 30: Horizontal, vertical trajectory , the ninth harmonic of an undulator with field errors

The field errors result in a spoilt trajectory with a remaining kick (to see it better, keep the left mouse button pressed on the 3d trajectory plot and move the mouse to rotate the plot).

In addition the on-axis spectrum of the flux-density is ruined (Fig. 30). But the later is mainly a consequence of the steering of the undulator, i.e. we can move the position of the observer to take this into account by looking for the point the trajectory points to:

OBS1Y=-8888.

OBS1Z=-8888.

This triggers a straight line fit of the trajectory in order to estimate the best observation point. We run WAVE and find the spectrum of Fig. 31. The peak of the ninth harmonic is now higher; however, a significant reduction of the flux-density and a distorted shape remains due to the field errors.

We have used Gaussian errors in this example so far. Normally, one would not accept magnets with errors above a certain limit, let's say one standard deviation. This can be done by setting

BCRANSIG=1.0

UBANSIG(1)=1.0

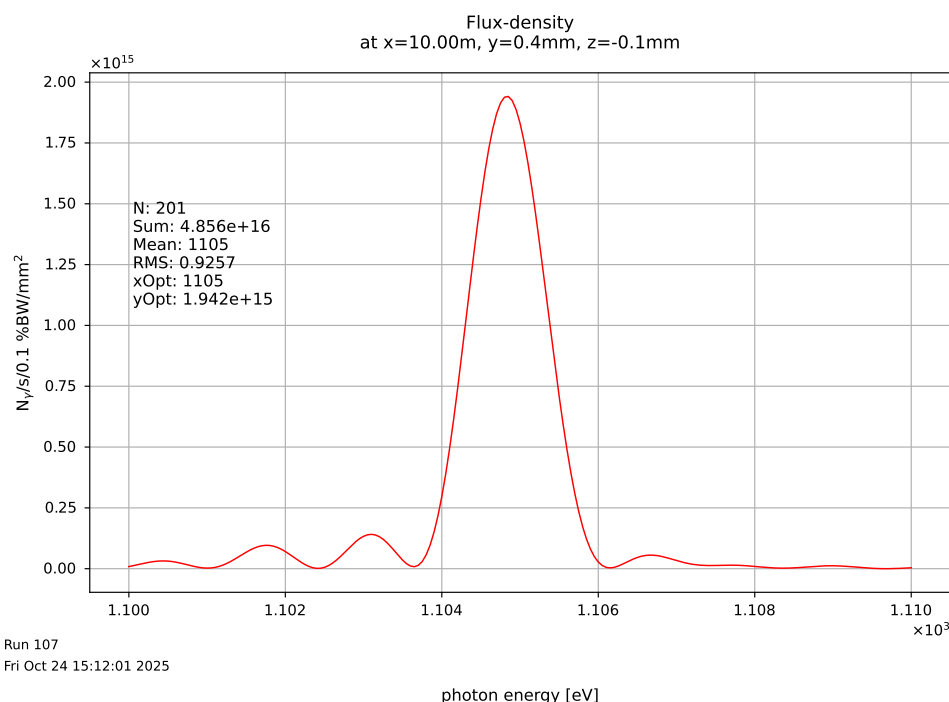


Fig. 31: Ninth harmonic at the shifted observation point

Run WAVE and enter in the plotting shell:

```
zone(2,2)
nz()
zone(2,2,3,'same')
ny()
zone(2,1,2,'same')
hcfluxden()
```

The result is shown in Fig. 32. We see a "realistic" trajectory and a good ninth harmonic.

These options allow the user to estimate the required quality of the magnets and trajectory of the undulator. There is another option **USIGOFFY(1)**, which limits the random walk of the trajectory due to angle errors of the magnets. This simulates the shimming procedure of the undulator and also allows to estimate the required quality of the trajectory.

However, these tools are provided to produce "realistic" trajectories and give hints about the impact of field errors. They do probably not replace a thoroughly analysis of the field measurement, analysis, and shimming of an undulator.

#### Proposed exercises:

Check the option **NURANMOD**. With **NURANMOD=2**, the error **BCRAN** can be applied to neighboring poles. This simulates a perfect shimming of the trajectory, while keeping the phase errors and there influence on the spectrum.

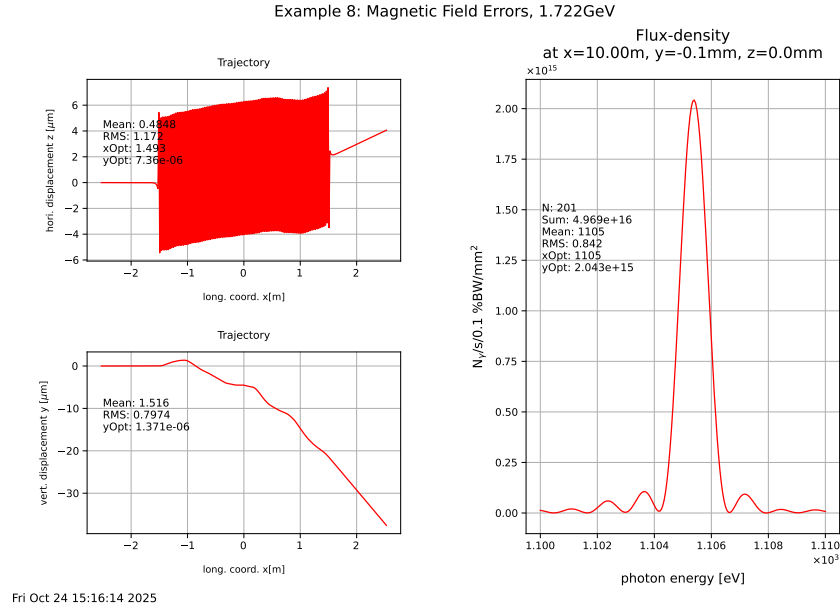


Fig. 32: Hori. and vert. trajectories and spectral flux-density of an undulator with limited errors

Dump the magnets of the model using *IKRESTOR*, change some magnet properties, and try to use the file to investigate the effects.

Check the option *IBERROR* and the namelist *\$BERRORN* and try to use it.

## 9. Example: Real Beams, Part II

In example 7, we've made the acquaintance of a delicate set-up. Due to the huge number of undulator periods, we are very sensitive to the input parameters of the beam. Therefore, the assumption, that the spectral and spatial distributions of the radiation is more or less the same for all electrons, is not valid anymore. Hence the folding procedures to take the energy-spread and emittance into account would fail. To overcome this difficulties, we must track many electrons according to the beam phase space distribution.

We consider a long beamline with 60 undulators. To calculate the spectrum of the spontaneous synchrotron radiation, we enter

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_9.py
cp $WAVE/input/wave_example_9.in wave.in
cp $WAVE/input/wave_example_9_magseq.in magseq.in
cp $WAVE/input/wave_example_9_wave.bmask wave.bmask
wave
wplot
```

This run takes some minutes, since for the considered 180 m long undulator section we need about a million integration steps for the spectrum calculation. For this single electron beam, we get a sharp peak for the on-axis flux-density (Fig. 33).

Example 9: Real Beams, Part II, 17GeV, 100.0mA

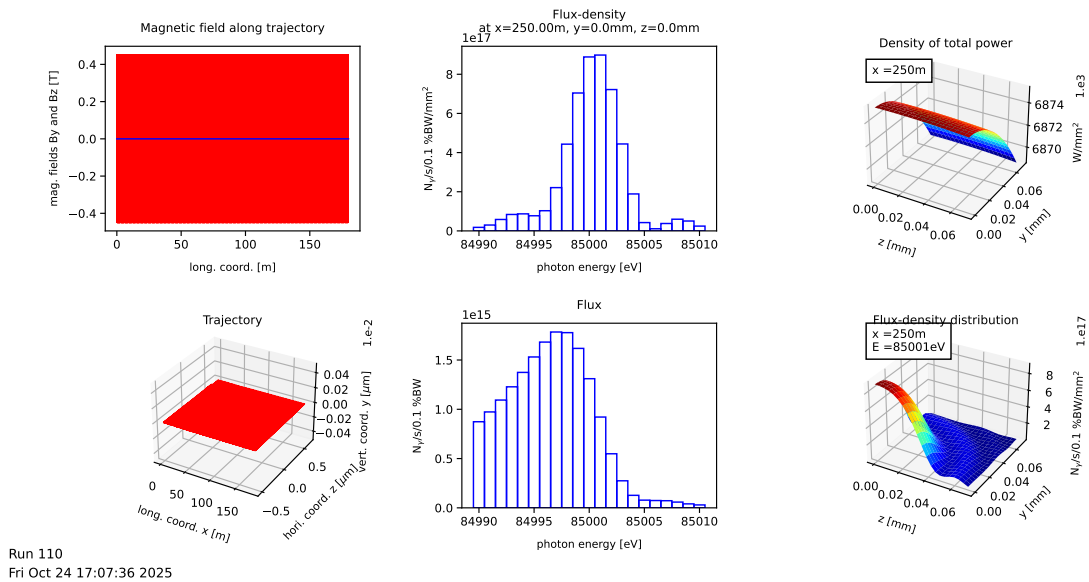


Fig. 33: 5th harmonic of a single electron beam for 30 undulator sections including continuous energy-loss and tapering of the undulators

The set-up of one six meter subsection section including two undulators is given in the file *magseq.in*. In combination with



```

XSTART=0.0
XSTOP=180.
IPERIODG=1
PERIODG=6.

```

this results in a beamline of 180 *m* length, where the field strength is corrected for the energy loss due to the spontaneous synchrotron radiation. Each section includes phase-shifters and quadrupoles to match the phase-advance and  $\beta$ -functions of the 30 sections.

Note, that for time reasons, only a quarter of flux is calculated, i.e. the pinhole is shifted, such that the lower left corner is on-axis. This is achieved by

```

IPBRILL=1
PINCEN(2)=9999.
PINCEN(3)=9999.

```

Now, we come to the actual task to calculate the 5<sup>th</sup> harmonic for a real beam, including energy-spread, emittance, and an energy-loss with quantum fluctuations. We start step by step setting in *wave.in*

```

IENELOSS=-1
IBUNCH=-1
NBUNCH=1000
NEINBUNCH=1
IPIN=0

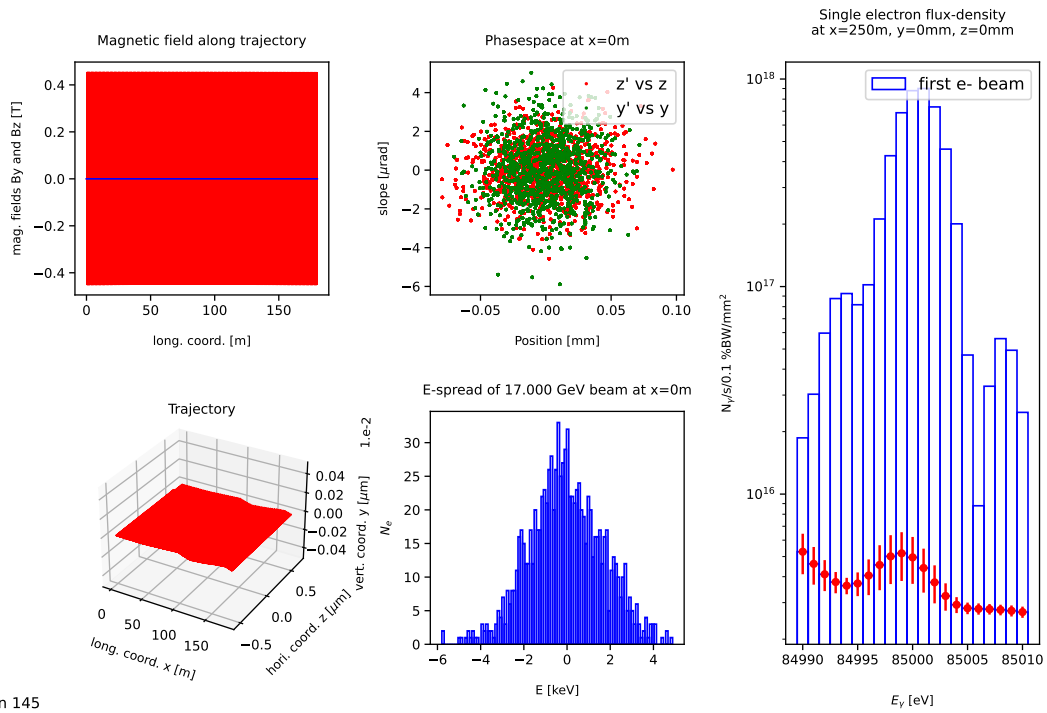
```

The variable *IBUNCH* triggers the bunch mode, i.e. to track a given number of bunches and electrons per bunch (*NBUNCH* and *NEINBUNCH* respectively). The radiation of different bunches is added incoherently, while the contributions of electrons of the same bunch are treated coherently. Thus, the setting given above calculates the radiation of 1000 electron, summed up incoherently. The resulting spectra are normalized to the beam current. Please, refer to the namelist *BUNCHN* for further options.

We run WAVE and get (after some minutes) the results shown in Fig. 34. We see some phase-space distributions of the beam electrons and the corresponding flux-density spectrum of a single electron beam in the lower right corner together with the same spectrum for the real beam.

We clearly see the flux-density reduced by the emittance and energy-spread. It reaches only about 0.5% of the original one.

Example 9: Real Beams, Part II, 17GeV, 100.0mA



Run 145  
Sun Oct 26 15:16:58 2025

Fig. 34: Flux-density of the 5th harmonic of 1000 electrons beam for 30 undulator sections including energy-loss with quantum fluctuations and tapering of the undulators (lower right plot)

### Flux through a pinhole

Since the calculation of radiation distributions in a pinhole are very time consuming, the mode **IPIN=3** can be used. In this mode, the radiation is not calculated for all grid points of the pinhole, but only for one randomly chosen point for each electron track. The idea is, that the random phase-space coordinates of the electron and the random observation point are not correlated, thus, for large enough a number of tracks, the flux trough the pinhole can be calculated by sampling the phase-space and the observation point simultaneously.

Edit **wave.in** and set:

```
IPIN=3
IPBRILL=0 PINW=0.00014
PINH=0.00014
```

and run WAVE and get Fig. 35.

Example 9: Real Beams, Part II, 17GeV, 100.0mA

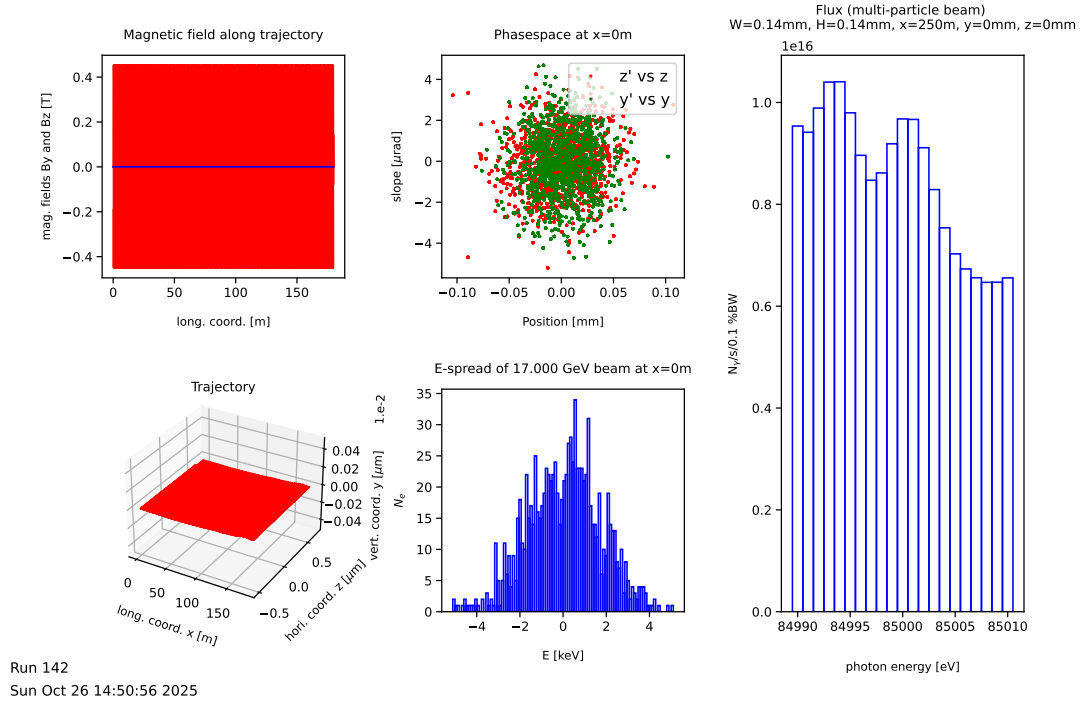


Fig. 35: Flux of the 5th harmonic of 1000 electrons beam for 30 undulator sections including energy-loss with quantum fluctuations and tapering of the undulators (lower right plot)

The flux is also significantly reduced compared to that of zero emittance and energy spread beam (Fig. 33).

#### Proposed exercises:

- Copy `$WAVE/input/wave.examples` to `wave.in`, run WAVE , and plot the results. Store the results and try to run WAVE in the multi-particle mode to compare with the previous results.
- Check the namelist `$BUNCHN` in `wave.in` and try to provide your own phasespace distribution.
- Check the namelist `$BUNCHN` in `wave.in`, and try to calculate coherent radiation of electrons.

## References

- [1] M. Scheer, "Beschleunigerphysik und radiometrische Eigenschaften supraleitender Wellenlängenschieber", Dissertation, Humboldt-Universität zu Berlin, Mathematisch-Naturwissenschaftliche Fakultät I, 2008, [urn:nbn:de:kobv:11-10096299](https://nbn-resolving.org/urn:nbn:de:kobv:11-10096299)  
M. Scheer, WAVE - A Computer Code for the Tracking of Electrons through Magnetic Fields and the Calculation of Spontaneous Synchrotron Radiation, <http://accelconf.web.cern.ch/AccelConf/ICAP2012/papers/tuacc2.pdf>
- [2] Michael Scheer, "UNDUMAG - a new computer code to calculate the magnetic properties of undulators", Proceedings of IPAC, Copenhagen, Denmark, 2017, pp 3071-3073